Ising Model

Jonas Al-Mersal

March 2026

1 The Ising Model

The ising model is a way to approximate physical systems of interacting particles. It is characterized by a Hamiltonean of the spin interaction and the magnetic field.

$$H = -\frac{1}{2}\sum_i \sum_{<i,j>} S_i \cdot S_j - \sum_i B \cdot S_i \tag{1}$$

Where <> denotes the sum over the first-neighbours. In this case we are going to investigate for the case where there is a big anisotropy and so $S = \pm 1$. We expect to observe the phase transition predicted for $k_B T_c = 2.269$.

1.1 The Sequential Code

In order to observe the phase transition we implemented a simple metropolis monte carlo method.

We define a square lattice with first-neighbour interactions and periodic boundary conditions.

Using the aforementioned hamiltonean, we calculate the energy difference from flipping a specific spin and then accept it following the probability

$$P(\Delta E) = min\{1, exp(-\Delta E/k_B T)\} \tag{2}$$

We implement a checkerboard pattern sweeping. On every Monte Carlo step we first flip the spins that have even $(x, y) \to x + y$ in the square lattice, and then the odd ones, this way we guarantee that we are not changing spins that are used by other calculations, avoiding race conditions in further parallelizations.

We finally calculate the average magnetization and the energy as checksums.

1.2 Results and Sanity Checks

In this subsection we investigate the results of our sequential code with respect to their physical sense and consistency. We plot the average Magnetization $\langle |M| \rangle$ of the system as a function of $k_B T/J$ in Fig. 1 to showcase the phase transition from uniform spins (up or down;$\langle |M| \rangle = 1$) to random spins ($\langle |M| \rangle = 0$). The code behaves as expected, since the increase in $k_B T/J$ adds thermal fluctuations which hinder the alignment of the spins to the applied magnetic field. In addition, increasing the strength of the magnetic field to $B_z/J = 0.05$ keeps spins aligned at higher temperatures compared to the case of $B_z/J = 0.01$, preserving ferromagnetic behaviour.

In low temperatures and high magnetic fields, all spins should align, reaching an average absolute Magnetization of 1. In order to test whether the system also behaves physically as expected in that case, we run a simulation of the sequential code for low temperature ($k_b T/J = 0.0001$) and strong magnetic field ($B = 1$). We plot the spins in the initial time of the simulation, as well as the final snapshot after the simulation has finished, in Fig. 2. We find that the initial state is completely random, while the final state is indeed completely ordered.

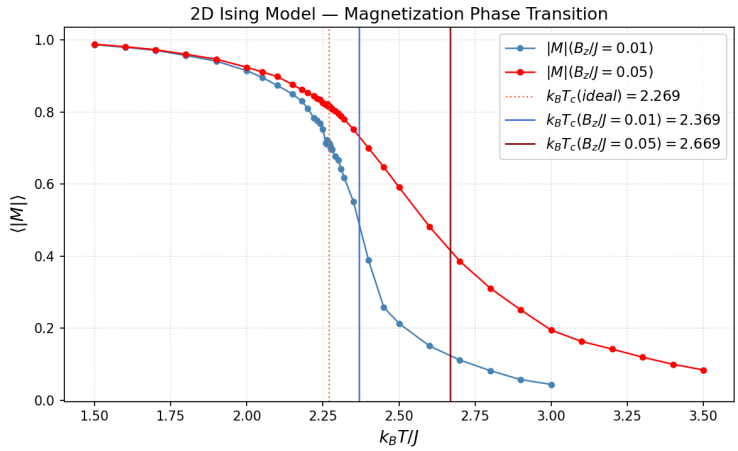

Figure 1: Plot of the final average magnetization $\langle|M|\rangle$ of the system (y axis) as a function of k_BT/J, for two different magnetic field strengths B_z/J as noted in the legend. We overplot the Temperature for which the phase transition occurs (solid lines), as well as the theoretically expected value for $B_z/J = 0$ (dashed line). Steps=10000, size=256x256.

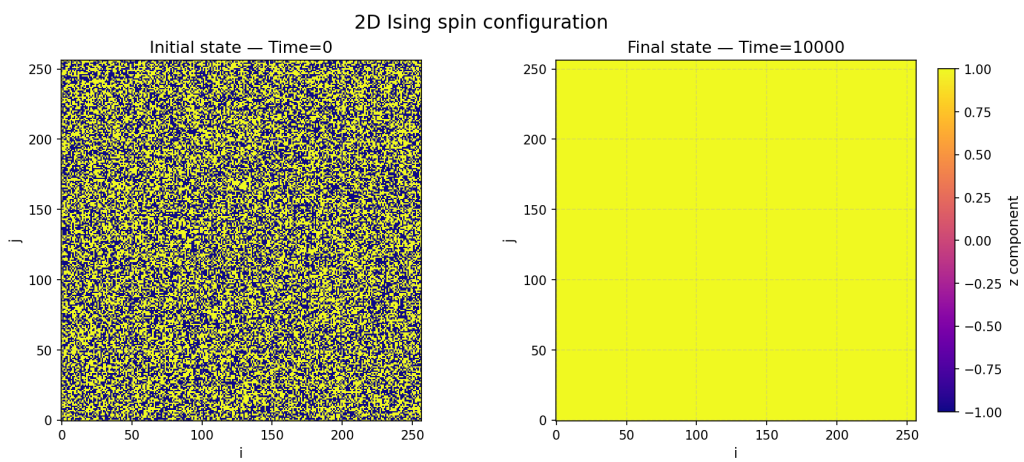

Figure 2: Initial (left) and final (right) spin-grid snapshots from the sequential simulation. The colourmap corresponds to up (yellow) and down (blue) spins.

We finally plot the total energy of the system vs. simulation time in Fig. 3. Starting from a random initial lattice, the energy rapidly decreases as the system tends to move to lower energies in order to reach equilibrium. This state corresponds to complete alignment towards magnetic field, as discussed above.

The above diagnostics are consistent with one another, confirming that the code is physically correct.

For the rest of the benchmarks we will be using a point close to the phase transition, which represents the most chaotic configuration of the system. This is done because if the checksums are correct in that point of the parameter space, then we can be sure of the correctness of our parallelization attempts. As a reference point, we also plot the initial and final states of a 256x256 grid in Fig. 4, for $k_BT/J = 2.35$ and $B_z/J = 0.01$.

2 Optimization using CUDA

The main point is that using CUDA we can initiate a huge number of threads, practically covering the whole grid with one thread per particle. The threads can then execute the half-sweeps in parallel, rather than one sweep being done sequentially in the CPU. Of course, the threads have to be synchronized correctly because

of the neighbor dependencies; the implementation has to be done carefully. The checkerboard pattern update is preserved in all CUDA implementations.

2.1 CUDA implementation (CUDA-v1)

The first CUDA implementation introduces several changes compared to the sequential code. In the sequential version, all functions are run on the host CPU and the sweep functions iterate and update the atoms class (atoms.z) in place. In the first CUDA version, sweeps become kernels and the loop index is replaced by a thread global index. In the sequential code we have an object (sys) which contains most of the parameters used. However it is not optimal performance-wise to pass this object into the GPU due to the object being made out of pointers to host memory. For that, we implement a structural change in the sweep functions, in which all necessary parameters are passed explicitly as variables. The total energy increment is no longer a plain host variable which is updated serially; it becomes a device-accumulated variable (dEtot) which is updated through atomic operations (atomicAdd), because many threads may accept spin flips concurrently. The control structure in the MonteCarlo function also changes substantially. We add memory allocations for all variables necessary, from host to device and we initiate the time loop. In the time loop, an even kernel is launched and then an odd kernel is launched. After one sweep we do not need to synchronize the device, because the device synchronizes automatically after the shutdown of a kernel. All arrays are then updated solely on the GPU across all timesteps. We add checkpoints in which we synchronize the device in order to copy back the values to the host and then to an output file. The checkpoints are added so that we do not have to access the CPU memory during all iterations, thus improving performance substantially.

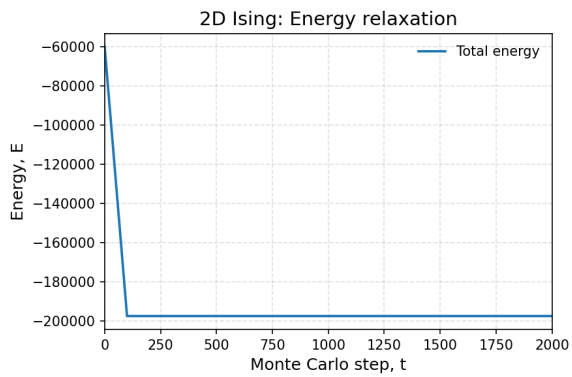

Figure 3: Total energy E as a function of Monte Carlo time t in sequential mode.

Viewed as a flow chart workflow, one full CUDA run follows this sequence: create lattice on host, compute initial host reference energy, allocate device arrays (dz, RNG states, checksum accumulator), copy initial spins from host to device, initialize per-site RNG states with the InitRng function on the GPU, enter the timestep loop, launch even sweep kernel, launch odd sweep kernel, copy intermediate checksum for logging, and repeat for all timesteps. We synchronize at the end, copy final spins and checksum back to host, free device memory, recompute final host energy from copied spins for validation, then print timing and observables.

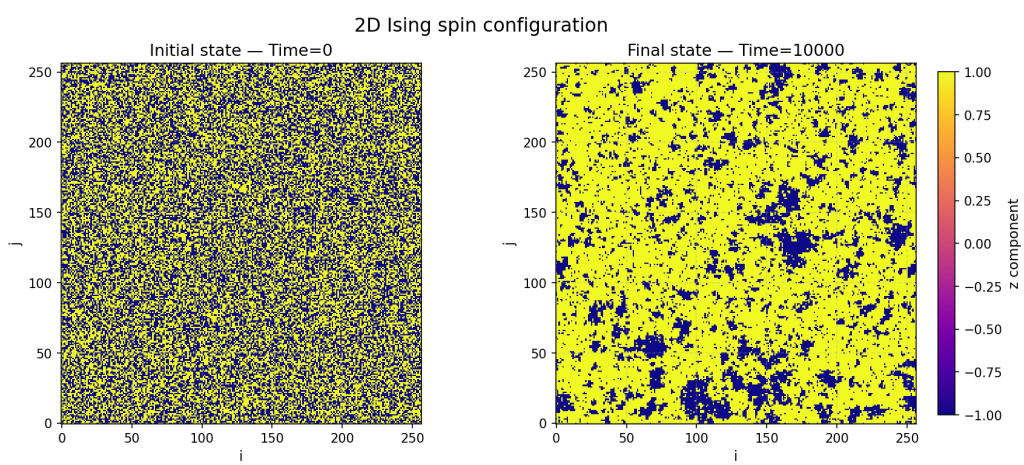

Figure 4: As Fig. 2, but for $k_BT/J = 2.35$ and $B_z/J = 0.01$.

2.2 Working on shared memory tiles (CUDA-v2)

The calculations in the Ising model are not particularly heavy, once the first layer of parallelization is implemented; the model is memory-bound. Thus, the next step in our optimization implementation is doing calculations in shared memory which can be accessed substantially faster. For that, we keep the same architecture but we change how the neighbor data is accessed. Instead of every thread reading all neighbors directly from global memory, each block loads a local tile of spins into shared memory, as well as a layer of ghost cells in order for the cells in the edge to have access to their neighbors. Then, each thread computes the local energies and spins from shared memory values. This reduces the number of global memory reads and should improve performance.

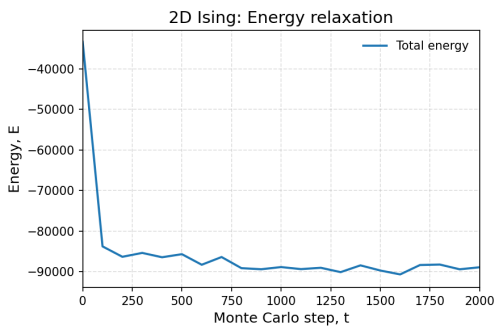

Figure 5: Total energy E as a function of Monte Carlo time t in parallel mode ($k_BT = 2.35$ and $B_z/J = 0.01$).

There are some important things to be careful about in this implementation. First, we have to handle the indexing very carefully between global coordinates, block coordinates and thread local coordinates, since one wrong calculation or offset would give access to the wrong neighbor and thus the wrong energy. After the tile and ghost cells load, all threads need to be synchronized using `__syncthreads()` so that all threads see a complete local tile. Since each calculation of the checkerboard pattern is completely independent within an even/odd sweep, and the device is automatically synchronized between the sweeps of different parity, there is no need to use `__syncthreads()` after every tile calculation. Finally, checksum accumulation should still be done in a block-local way, then atomic add to a global accumulator to avoid race conditions and reduce unnecessary atomic contention.

In the actual shared-memory CUDA code, tile geometry is fixed with defined constants (using the `#define` function) and those constants are used to size and index the local cache. Shared-memory arrays are declared explicitly with `__shared__`. The kernel then fills interior cells and ghost cells before calling `__syncthreads()` to guarantee correctness. The atomicAdd function is still used to update the global checksum (dEtot), but this operation remains a bottleneck because many blocks may contend on the same memory location. In practice, checksum loging is a debugging feature that is not required for our runs, after we validate that the model is indeed correct. The checksum accumulation can be disabled by compiling without 'CHECKSUM' (all `#ifdef CHECKSUM` sections are skipped), which removes most atomic overhead and can improve runtime.

2.3 Changing variable types (CUDA-v3)

Compared to the previous shared-memory version, we also explore the effect of the types of variables to the runtime of our code. We use `'signed_char'` instead of wider integer types, while all `double` type variables are turned into `float`. As said above, this code is memory-bound; in this way, we reduce the size per element that travels across memories, increasing the amount of data that can be moved and stored simultaneously. Nevertheless, a caveat of numerical precision is introduced and so the checksums are marginally different between higher precision runs. This is the final and possibly most optimized version of our code.

2.4 Results

As shown in Table 1, the checksums are correct between the initial and final states of the systems of the all parallelized versions. For consistency, we also repeat Fig. 3 for the parallel version of our code, for $k_BT = 2.35$ and $B_z/J = 0.01$ in Fig. 5.

2.5 Sanity checks

After confirming the correctness of the physical behaviour of our parallelized code, in this subsection we investigate whether the checksums are correct for every version. We note that due to library differences between CPU and GPU, the seeds of the random generators are different and so the checksums are expected to be marginally different ($\sim 1\%$) in every run. The above is increased by the fact that floating point addition is non-associative; the order of sums in atomicAdd is not guaranteed to be the same between runs. Finally, we declare that v2 and v3 versions of the code, use a constant seed for the random generator (equal to the thread ID).

2.6 Profiling

After implementing all versions we ran the nsys profiler to analyse how the vGPU runtime is distributed. As shown in Table 2, kernel launching dominates across all versions, suggesting that the separate red/black checkerboard kernel launches introduce non-negligible overhead. We also observe notable differences in memory allocation and copy contributions between V1 and V2/V3, despite checksums being disabled throughout. We attribute this to differences in memory access patterns: V1 assigns a block of 256 threads to a row of 256 contiguous spins, while V2 and V3 read 16×16 tiles, which may affect how data is staged on the GPU.

Version	E_{init}	ΔE	ΔE_{acc}	$\langle \lvert M \rvert \rangle$
Seq	859.4	−88254.3	−88254.3	0.5598
CUDA-v1	859.4	−89066.4	−89066.4	0.5661
CUDA-v2	859.4	−89248.7	−89248.7	0.5757
CUDA-v3	859.402	−89462.3	−89469.2	0.5761

Table 1: Initial energy E_{init}, total energy difference ΔE, accumulated energy difference ΔE_{acc}, and mean absolute magnetization $\langle \lvert M \rvert \rangle$ per code version. The discrepancy in CUDA-v3 is due to `float` precision.

Metric	**V1 (Parallel cuda)**	**V2 (Shared Mem)**	**V3 (Simple Types)**
CUDA API calls (% of total API time)			
cudaLaunchKernel	61.3%	68.1%	68.1%
cudaMalloc	27.5%	18.2%	18.7%
cudaMemcpy	0.0%	13.2%	12.7%
cudaDeviceSynchronize	11.1%	0.4%	0.4%
cudaFree	0.1%	0.1%	0.1%
cudaMemset	0.0%	0.0%	0.0%
GPU kernels (% of GPU kernel time)			
SweepOdd / SweepEven	49.8% / 49.3%	—	—
SweepAll	—	99.5%	99.5%
InitRng	0.7%	0.4%	0.4%
Etot + kern_Mag	0.1%	0.1%	0.1%
Total GPU kernel time			
Time (s)	0.289 (48.3%)	0.494 (62.1%)	0.503 (62.1%)

Table 2: NSYS profiling summary for the three CUDA Ising implementations at $L = 256$, $N_{\text{steps}} = 3000$.

2.7 Performance comparison

After verifying that the implementations are correct physically and looking at possible overheads, we set a number of steps and ommit the checksums in the code in order to benchmark the performance. We first try to verify at what point do we start getting asymptotic performance for the lattice size. For this we plot the amount of time needed to update a single spin on each step in Figure 6. There we can see that for $L^2 \sim 10^4 - 10^5$, we start getting an asymptotic behaviour, so we decide to select $L = 2048$ for that type of comparison. We also tested out using a smaller block size of 64 threads as that would allow for only 32 threads making calculations, which should move in lock-step and so have less latency in syncing.

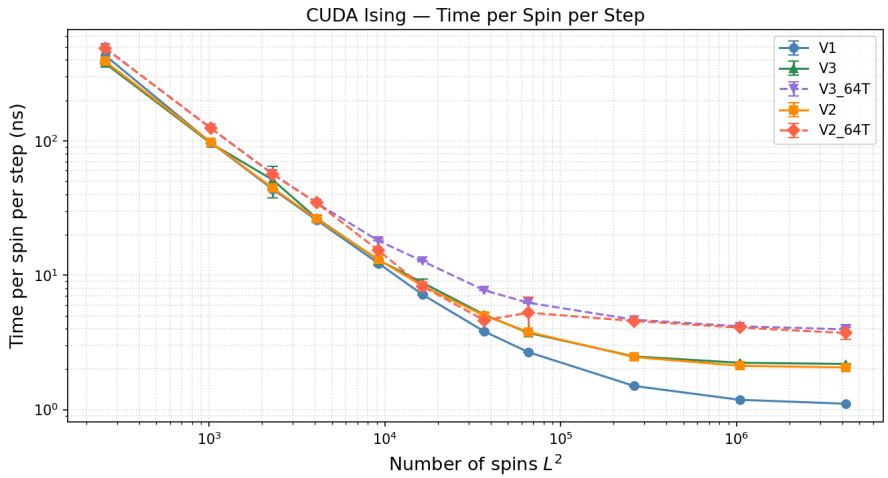

Figure 6: Per flip performance of the cuda implementations, k_BT=2.35, B=0.01, computed from wall time for 3000 MC steps

Furthermore this limit is close to the $\approx$ 40000 concurrent threads in our vGPU. Since the calculations are so minimal, we expect each thread to be done quite quickly andd so increasing the number of spins allows for better latency hiding. Before reaching the asymptotic behaviour, we get a big overhead from launching the kernels for each iteration since the time spent inside the kernel is minimal.

We also plot the amount of time per step in order to have some degree of comparison for different system sizes with the sequential code, in Fig. 7.

Here we notice how the performance for the smaller block sizes using only 64 threads is significantly worse. That might be due to requiring a larger coverage of ghost zones in order to perform the calculations. One possible fix for this would be to have threads perform more than 1 calculation, this way we might be able to compute larger tiles using the same block size, while also increasing the workload per thread as that seems to be a limiting factor.

Finally, we showcase in Table 3 the results of asymptotic runtime for the different versions. As we can see, the first CUDA implementation ended up being the fastest one. that is because the requirement for the calculations is not that big to justify using shared memory in the way it is done currently. each spin is accessed 4 times by others.

Implementation	**Speedup vs Sequential**
V1	25.7×
V2 (16×16 tile)	13.8×
V3 (16×16 tile)	13.0×
V2_64T (8×8 tile)	7.6×
V3_64T (8×8 tile)	7.2×

Table 3: CUDA speedup over sequential at $L = 2048$ (3000 MC steps).

From this whole benchmark we notice that the performance is significantly improved, however the shared memory implementation and the changes in block size seem to make it worse.

This implies that the overhead from loading ghost zones does not outweigh having to load spins only once instead of 4 times. Since each block loads around 256 spins, which corresponds to something close to 2kB, it might fit into the L1 cache, which can hold up to 512kB. For that reason, using shared memory might not be that much faster.

One way to make better use of shared memory would be to load larger tiles for each block. We could also try making better use of threads, making sure we do not have threads that load and don't compute spins due to the checkerboard pattern.

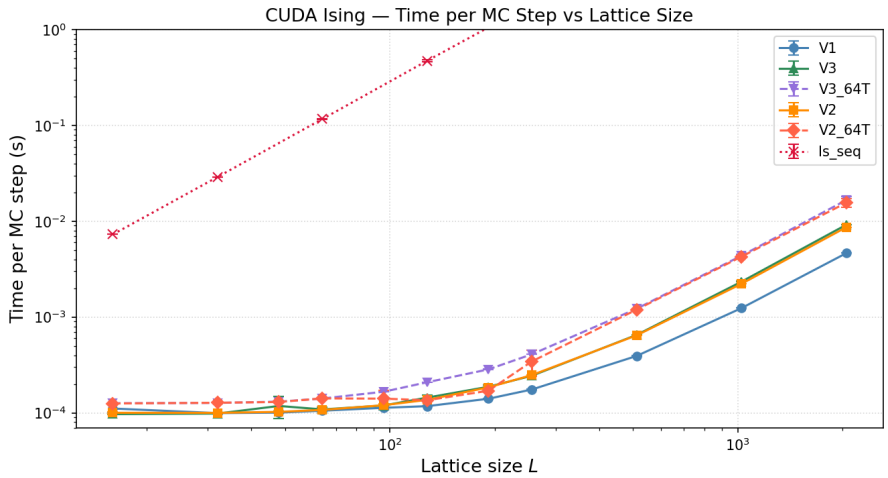

Figure 7: Per step performance of the cuda implementations, k_BT=2.35, B=0.01, computed from wall time for 3000 MC steps

One final implementation that could have been done but was left out would be to store spins as simple bits, which would make loading a lot faster.

3 Optimization using OpenMP and MPI

OpenMP and MPI are two powerful methods used to parallelize the program and achieve speedup relative to the sequential code. We first created a parallel program using OpenMP in which the lattice was shared across multiple threads. This was achieved by inserting various pragmas into key functions of the code. We also implemented another version, which in addition to OpenMP used MPI to split the lattice across processes. The checkerboard logic remained as in the sequential version.

3.1 OpenMP

In the OpenMP implementation, we used multiple threads operating in shared memory. We started by parallelizing the construction of the lattice using a `schedule(static)` pragma that distributed the construction of neighbor lists across threads. The core functions that needed to be parallelized were the Monte Carlo sweeps. In these functions we added the pragma: `#pragma omp parallel for collapse(2) schedule(static) reduction(+:Etot)`. Here `collapse` was used to merge the two loops over lattice indices into one iteration space, improving load balancing across threads. Another important part of our implementation was that we ensured each thread has its own local random number generator as opposed to a global one, so we could avoid contention between threads and ensure each thread is independent. We also used parallel reduction pragmas for the calculation of the total energy and magnetization.

3.2 OpenMP & MPI

We also extended the OpenMP implementation by creating a hybrid OpenMP & MPI implementation. Here, apart from the changes discussed in the OpenMP subsection we made some changes suitable for MPI. Firstly, we made a domain decomposition of the lattice across one direction, so the lattice is split across multiple MPI processes. Each process handles a subset of the rows. Furthermore we introduced ghost cells, to ensure the correct computation of neighbor interactions at the boundaries. Communication is achieved using `MPI_Sendrecv`. Then, the total energy and magnetization was computed globally using `MPI_Allreduce` which combines contributions from all the processes. Finally the full lattice is reconstructed using `MPI_Gatherv` because different processes may store different number of rows.

Code Version	E_{init}	ΔE	ΔE_{acc}	$\langle \lvert M \rvert \rangle$
Seq	859.4	-88513.2	-89372.6	0.575623
OpenMP	533.16	-88326.9	-88860.1	0.578217
OpenMP + MPI	42.72	-88994.5	-89037.2	0.608002

Table 4: Initial Energy, Energy difference between final state and initial state (ΔE), the ΔE computed by accumulating all changes in Energy due to each individual spin-flip (ΔE_{acc}, as well as the average absolute Magnetization $\langle \lvert M \rvert \rangle$ for every code version.

3.3 Ghost cells and boundary exchange

To handle neighbour interactions at the boundaries between MPI ranks, each process stores two extra ghost rows: one above its local block and one below it. These rows are not owned by the process itself, but contain copies of boundary data received from neighbouring ranks. The ghost rows are updated using MPI_Sendrecv, where each rank sends its first owned row to the rank above and its last owned row to the rank below, while receiving the corresponding boundary rows into its ghost cells. This makes it possible to compute the interaction energy correctly even for spins located at the edge of a local subdomain.

3.4 Maintaining the even-odd update scheme across MPI processes

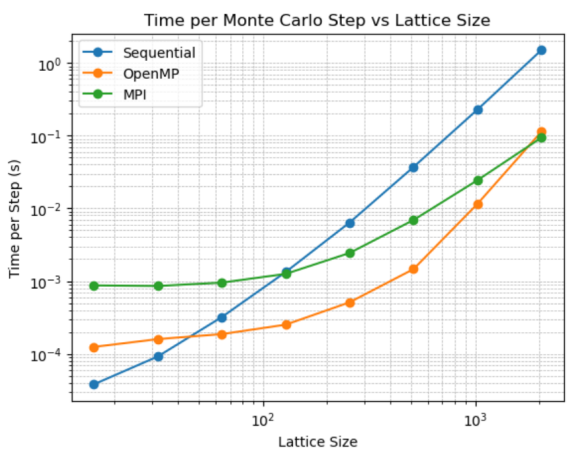

Figure 8: Time per Monte Carlo step as a function of lattice size L for three parallelisation strategies: sequential, OpenMP, and MPI. Both axes are on a logarithmic scale.

The row-wise MPI decomposition introduces a small but important issue: each rank stores only a local block of the lattice, so the local row numbering restarts from zero on every process. This matters because the even-odd checkerboard update must remain consistent across the full lattice; otherwise, neighbouring ranks could apply different parity rules at the subdomain boundaries. To avoid this, each rank stores the global index of its first owned row and reconstructs the global row number during the sweep, so that the parity test is always applied in global rather than local coordinates.

3.5 Sanity Checks

In order to verify that the parallel versions produce correct results we verify that physical observables are consistent across implementations. We observe that the energy decreases rapidly and then stabilizes, indicating that the system approaches equilibrium. The final energy is indeed the same in all versions. The same is true about magnetization. The values are not exactly the same but they are very similar as expected by the stochastic logic of the Monte Carlo simulation. The fact that the final energy and magnetization is the same across the sequential, OpenMP and hybrid(OpenMP + MPI) versions confirms that all implementations are physically consistent.

3.6 Results

We benchmarked the performance of the implementations using different lattice sizes and 200 Monte Carlo steps. Using 64 threads in the OpenMP version and 2 nodes, each containing 32 threads in the MPI version we obtained the following results:

Size	Sequential (s/step)	OpenMP (s/step)	Hybrid OpenMP+MPI (s/step)
16	3.80e-05	1.24307e-04	8.73175e-04
32	9.18e-05	1.59128e-04	8.56495e-04
64	3.19e-04	1.86538e-04	9.52030e-04
128	1.33e-03	2.51301e-04	1.25466e-03
256	6.38e-03	5.10270e-04	2.43392e-03
512	3.70e-02	1.46080e-03	6.91625e-03
1024	2.28e-01	1.15145e-02	2.43588e-02
2048	1.50e+00	1.13376e-01	9.37175e-02

Table 5: Time per Monte Carlo step for sequential, OpenMP, and hybrid OpenMP+MPI implementations.

As shown in Figure 8, the OPENmp implementation achieves the best performance for large lattice sizes, while OpenMP+MPI outperforms the sequential baseline across all sizes tested. However, for size=2048 we can see than hybrid version starts to perform better than clear OPENmp.

3.7 Profiling

To complement the weak-scaling and timing results, we also profiled the three implementations in order to identify the dominant sources of runtime and to examine how the performance bottlenecks change with the parallelisation strategy. For the sequential version, `gprof` produced a meaningful function-level breakdown, clearly identifying the main computational hotspots. The same approach was attempted for the OpenMP version, but the reported dominant functions were not physically consistent with the structure of the simulation, which is a known limitation of `gprof` for multithreaded OpenMP codes. For the hybrid MPI+OpenMP version, the profiling results were again more plausible and gave a useful view of the dominant costs, particularly for the measured rank-0 process. To enable comparison across implementations, the profiler output was grouped into broader operation categories rather than individual function names.

Operation	V1 (Sequential)	V2 (OpenMP)	V3 (MPI+OpenMP)
Energy evaluation	45.99%	–	38.38%
Random number generation	42.34%	–	48.23%
Sweep / MC control	8.03%	–	0.00%
Gather / data collection	–	–	13.38%
Ghost exchange	–	–	0.00%
Auxiliary / other	3.65%	–	0.00%
Total profiled time	1.37 s	1.37 s	3.96 s

Table 6: Distribution of profiled runtime across major operation categories for the sequential, OpenMP, and MPI+OpenMP implementations. Percentages are reported as shares of total sampled profiler time.

The profiling results show that the sequential code is dominated by two main costs: local energy evaluation and random number generation. Together, these account for almost the entire sampled runtime, indicating that the Monte Carlo kernel is primarily compute-bound and stochastic-sampling-bound. The sweep functions themselves contribute a much smaller direct share, which suggests that most of their cost is spent in the lower-level routines they invoke rather than in the sweep control logic itself.

For the hybrid MPI+OpenMP implementation, the same two dominant patterns remain visible: energy evaluation still represents a major fraction of runtime, while random number generation becomes even more prominent. In addition, a non-negligible overhead appears in `GatherLattice`, showing that distributed-memory data collection introduces an extra performance cost that is absent in the purely sequential version. By contrast, ghost-zone exchange appears with negligible direct cost in the measured profile, at least for rank 0. The OpenMP-only profiling results are not interpreted quantitatively in the table, since the reported hotspots were dominated by compiler-generated or setup-related functions rather than physically meaningful

computational kernels. Overall, the profiling analysis confirms that the core performance bottlenecks of the application are the energy computation and the random-number generation, while the hybrid parallel version introduces an additional data-gathering overhead associated with the MPI decomposition.

4 CUDA vs OpenMP (+MPI)

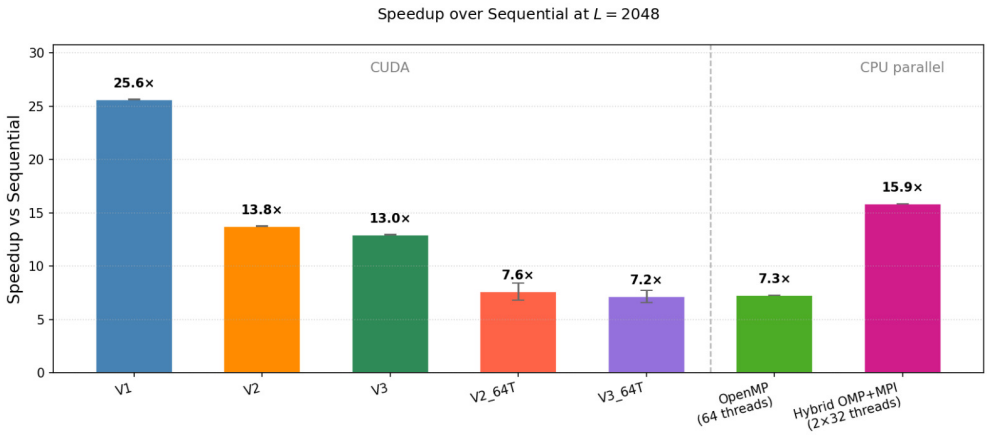

Figure 9: Speedup relative to the sequential implementation at $L = 2048$ for the different CUDA and CPU-parallel versions.

Figure 9 provides a direct comparison of the best speedups obtained by the GPU-based and CPU-based parallel implementations for the largest benchmarked lattice, $L = 2048$. The most important point to note is that the CUDA results were obtained using only one quarter of a GPU, whereas the OpenMP result uses an entire compute node with 64 CPU threads and the hybrid OpenMP+MPI result uses two nodes with 32 threads each. Therefore, the comparison is not only between programming models, but also between different levels of hardware allocation.

Within this benchmark, the GPU implementations clearly deliver the highest absolute performance. In particular, CUDA-v1 reaches the best overall result, with a speedup of 25.6× over the sequential baseline. The remaining CUDA variants still outperform or remain competitive with the CPU-parallel implementations, but the shared-memory versions do not improve on the simpler first CUDA implementation. This is consistent with the profiling results and with the memory-access pattern of the Ising update: each spin interacts only with a small number of neighbours, so the extra overhead of shared-memory tiling is not sufficiently compensated by data reuse. In addition, reducing the block size to 64 threads further decreases performance, most likely because it increases synchronization and tile-boundary overhead while reducing the effective amount of useful work per block.

On the CPU side, OpenMP reaches a speedup of 7.3×, while the hybrid OpenMP+MPI implementation reaches 15.9×. This shows that distributed-memory parallelism becomes beneficial at large lattice sizes, where the computational workload is sufficient to amortize the communication and gathering costs. Even so, the hybrid CPU result remains below the best CUDA performance, despite using substantially more total hardware resources.

Overall, the results indicate that, when GPU resources are available, CUDA is the most effective strategy for this application. At the same time, the CPU implementations remain highly relevant in practice. Both approaches are scalable, but GPU resources are often more limited than CPU nodes in shared HPC environments. For this reason, if hardware availability is constrained, OpenMP or hybrid OpenMP+MPI may still be attractive alternatives, since they achieve strong speedups without falling dramatically behind the GPU implementations.